

Navigation and listings

Sidebar

Navigation can be explicit:

```
[sidebar]

[[sidebar.section]]
title = "Guide"

    [[sidebar.section.item]]
    path = "index.typ"
    label = "{icon:home} Home"

    [[sidebar.section.item]]
    path = "usage.typ"
    label = "Usage"
```

Each section can contain any number of `item` entries. An item can point to one file with `path`:

```
[[sidebar.section.item]]
path = "install.typ"
label = "{icon:lucide:download} Install"
```

An item can also include files with a simple glob:

```
[[sidebar.section.item]]
glob = "guide/*.typ"
```

If no sidebar is configured, *Calepin* builds navigation from `.typ` files in the source directory. Hidden paths are skipped by default. Set `show_hidden = true` under `[sidebar]` to include hidden paths in manual navigation file discovery.

Sidebar entries are not the full build list. Use `[pages].include` in `calepin.toml` for pages that should be rendered but not linked from navigation.

In multilingual sites, sidebar entries are resolved separately inside each configured language `content_dir`. Generated navigation only shows entries for the current page's language.

Navbar

The top navigation bar can be configured with three regions:

```
[navbar]

[[navbar.item]]
position = "left"
path = "index.typ"
label = "Home"

[[navbar.item]]
position = "center"
path = "install.typ"
label = "Install"

[[navbar.item]]
position = "right"
```

```
url = "https://github.com/user/repo"
label = "{icon:simple-icons:github} GitHub"

[[navbar.item]]
position = "right"
widget = "language"

[[navbar.item]]
position = "right"
widget = "theme"
```

Navbar entries use the same `path`, `glob`, and `label` fields as sidebar entries. A navbar entry can also use `url` for an external link, or `widget` for a built-in control. `position` can be `left`, `center`, or `right` and defaults to `left`. Pages referenced only from the navbar are still included in the website build.

Use `path` for one local page, `glob` for several local pages, and `url` for an external destination. Local page links are rewritten relative to each generated page, and external URLs are passed through unchanged.

In multilingual sites, local navbar links are language-aware and external navbar links are global. This means a `url` item can appear for every language, while local page entries are filtered to the current page's language.

Widget names are open-ended theme hooks. The bundled themes understand `widget = "theme"` for the light/dark/auto selector and `widget = "language"` for the language picker when more than one language is configured. Custom themes can define their own widget names, such as `widget = "search"` or `widget = "profile-menu"`, and render them by checking `item.widget` in the theme template. A widget item cannot also set `path`, `glob`, or `url`. When `[navbar]` is omitted, the bundled website themes include the default theme and language widgets on the right. When `[navbar]` is configured, include these widget items explicitly wherever you want them.

Navigation icons

Sidebar and navbar labels can include icon tokens with `{icon:prefix:name}`. Icon names use Iconify syntax, such as `{icon:luclide:download}` or `{icon:simple-icons:github}`. If the prefix is omitted, *Calepin* uses `luclide`, so `{icon:home}` means `{icon:luclide:home}`.

Calepin downloads icons from Iconify during the build with a short timeout, stores them under `.calepin/icons/`, and inlines the cached SVG in generated HTML. Later builds use the cache without downloading again. To render an icon without visible text, use a label that contains only the icon token, such as `label = "{icon:simple-icons:github}"`; *Calepin* keeps the resolved page title or URL as the accessible label. For compatibility, an item can also use `icon = "luclide:home"` as shorthand for prepending an icon to the resolved label. *Calepin* does not grant rights to third-party icons; check the license and trademark rules for the icon sets you use.

Page titles

Navigation labels are resolved in this order:

1. the `label` configured on the sidebar or navbar item
2. the `title` entry of the page's `<website-metadata>` metadata
3. the file name, with `-` and `_` replaced by spaces

To set a page title in the document itself:

```
#metadata((title: "Getting started")) <website-metadata>
```

Page metadata is extracted while the page is preprocessed, from the same staged source used for rendering. Package imports such as `@preview/calepin` are already rewritten at that point, so metadata works in pages that do not compile standalone.

Page listings

Pages can list other pages of the website, for example to build a blog index. During a website build, every page can call `calepin.pages()`:

```
#import "@preview/calepin:0.0.1" as calepin

#for post in calepin.pages()
  .filter(p => p.path.starts-with("blog/"))
  .sorted(key: p => p.meta.at("date", default: ""))
  .rev() [
- #link(post.href)[#post.title] (#post.meta.at("date", default: ""))
]
```

Each entry is a dictionary:

- `path`: the source file, relative to the website source directory (`blog/first.typ`)
- `href`: a link to the page, already relative to the current page
- `title`: the resolved display title, matching the navigation
- `language`: the configured language code, or none for a single-language site
- `translation_key`: the key used to connect translations of the same page
- `translations`: a dictionary mapping language code to that translation's generated href
- `pdf`: a link to the page's PDF, or none when no PDF is rendered
- `meta`: the page's `<website-metadata>` dictionary, verbatim

Calepin interprets only the `title`, `pdf`, `hidden`, `slug`, `url`, and `translation_key` keys of page metadata. `slug` renames the generated file inside the page's directory, and `url` sets the full output path relative to the site root; both stay inside the output directory. Everything else in `meta` (dates, tags, authors, categories) is yours to define and filter on. Outside a website build, `calepin.pages()` returns an empty array, so pages compile standalone.

Blog posts usually should not appear in the sidebar. Include them in the build with `[pages]`:

```
[pages]
include = ["blog/*.typ"]
```

Then mark each post:

```
#metadata((title: "First Post", date: "2026-06-01", hidden: true)) <website-
metadata>
```

`hidden: true` removes a page from automatic navigation. Pages included through `[pages].include` are built, listed in `calepin.pages()`, and present in `sitemap.xml` without needing to appear in the sidebar or navbar.

When any page's metadata changes, *Calepin* rebuilds the whole site during watch so that listing pages never go stale.